VIXOR

Refactor Plan

Comprehensive Sprint-by-Sprint Execution Guide

Based on Six Comprehensive Audits

Architecture | Product | Domain | Component | UI/UX | Technical

221 Total Problems Identified | 6 Sprints | 14 Weeks | 42 Core Tasks

Generated: July 2025

Table of Contents

1. Executive Summary

This Refactor Plan consolidates findings from six independent audits conducted across the VIXOR codebase, covering Architecture, Product scope, Domain model, Component design, UI/UX patterns, and Technical infrastructure. Together these audits identified 221 distinct problems ranging from critical blockers that prevent production deployment to low-priority polish items. The audit team classified each problem by severity: P0 indicates an immediate blocker that must be resolved before any further development, P1 represents significant issues that degrade developer experience or user trust, and P2 covers improvements that enhance maintainability or user satisfaction. The plan addresses every finding through 42 core tasks organized into six time-boxed sprints spanning 14 weeks, ensuring a methodical and risk-managed approach to resolving technical debt while preserving business continuity.

The most urgent findings are architectural in nature. The project currently lacks any continuous integration or deployment pipeline, meaning every merge to the main branch is deployed manually without automated testing, linting, or quality gates. Furthermore, the test suite is entirely absent, with zero unit tests, integration tests, or end-to-end tests, leaving every code change unverified against regressions. On the product side, the application exposes 39 distinct routes for what should be an MVP product, indicating severe feature creep that dilutes engineering focus and overwhelms new users. The domain layer, while cleanly structured as a directed acyclic graph, contains 23 distinct domain modules with 43 identified problems, including three P0 issues that affect data integrity. Component-level audits revealed that the AppShell component alone spans 1,688 lines of code, and seven chart variants exist without a unified abstraction, creating maintenance nightmares and inconsistent user experiences.

UI/UX audits uncovered 47 problems, including two critical accessibility violations that could expose the project to legal compliance risk under WCAG 2.1 AA guidelines. Mobile responsiveness gaps exist across multiple pages, and weak empty states leave users confused when no data is available. Technical infrastructure audits confirmed the absence of monitoring, observability, API versioning, and a fragile build configuration that breaks under minor dependency changes. This plan prioritizes developer productivity infrastructure first, then moves to component consolidation, domain cleanup, UX improvements, and finally feature rationalization, ensuring each sprint builds upon the foundations laid by its predecessors.

Audit Area	Total Problems	P0	P1	P2	Primary Risk
Architecture	38	2	8	28	No CI/CD, no tests
Product	47	3	18	26	39 routes for MVP
Domain	43	3	22	18	3 P0 data integrity issues
Component	18	0	10	8	AppShell 1,688 LOC
UI/UX	47	2	15	30	Critical a11y violations
Technical	28	2	12	14	No monitoring, no API versioning
TOTAL	**221**	**12**	85	124	

Table 1: Audit Findings Summary by Area and Severity

2. Sprint Overview

The refactor is organized into six sequential sprints, each with clear entry criteria, deliverables, and exit criteria. This phased approach minimizes risk by ensuring foundational infrastructure is in place before higher-level refactoring begins. Each sprint is designed to be independently valuable, meaning that even if the plan is paused after any sprint, the codebase is left in a better state than before. Sprint durations range from one week for the critical infrastructure sprint to four weeks for the feature rationalization sprint, reflecting the relative complexity and risk of each phase. Total estimated effort across all sprints is approximately 520 person-hours, which can be distributed across one to three engineers depending on team size and parallelization opportunities.

Sprint	Weeks	Focus Area	Tasks	Est. Hours	Key Deliverable
Sprint 0	1	Critical Fixes	8	**60**	CI/CD pipeline, test runner, shadcn fix
Sprint 1	2–3	Infrastructure	7	**90**	Monitoring, error boundaries, API versioning
Sprint 2	4–5	Component Refactor	8	**95**	Chart consolidation, AppShell split
Sprint 3	6–7	Domain Cleanup	7	**80**	Domain consolidation, dead code removal
Sprint 4	8–10	UX Improvements	6	**85**	Accessibility, mobile, empty states
Sprint 5	11–14	Feature Rationalization	6	**110**	39 routes reduced to 12 core routes

Table 2: Sprint Timeline and Resource Allocation

Sprint dependencies form a strict linear chain: Sprint 0 establishes the safety net of automated testing and CI/CD that all subsequent sprints rely on for verification. Sprint 1 builds observability infrastructure that enables data-driven decisions in later sprints. Sprint 2 consolidates components so that Sprint 3 domain cleanup operates against a stable UI layer. Sprint 4 UX improvements require the consolidated component layer from Sprint 2, and Sprint 5 feature reduction depends on knowing which domains are clean from Sprint 3. This dependency chain means parallelization between sprints is not recommended, though tasks within a sprint can often be parallelized across multiple engineers.

3. Sprint 0: Critical Fixes (Week 1)

Sprint 0 addresses the two most critical architectural deficiencies that render all other work high-risk: the absence of a CI/CD pipeline and the complete lack of automated tests. Without these two foundations in place, every subsequent refactoring step carries the risk of introducing silent regressions that go undetected until production. This sprint also resolves the broken shadcn/ui path alias, which causes import errors and prevents proper component resolution, and removes unused dependencies that inflate bundle size and create confusion about the actual technology stack. The sprint is designed to be completed in a single week by a single senior engineer, though two engineers working in parallel on CI/CD and testing respectively could reduce this to four days.

The exit criteria for Sprint 0 are unambiguous and verifiable: every push to the main branch must trigger an automated pipeline that runs linting, type-checking, unit tests, and a build verification. The test runner must be operational with at least 10 smoke tests covering the most critical user flows. The shadcn/ui path alias must resolve correctly, and the dependency tree must contain zero unused packages. These criteria represent the minimum viable safety net that makes all subsequent sprints safe to execute.

3.1 Sprint 0 Task Breakdown

ID	Task	Priority	Deps	Hours	Difficulty	Affected Files	Expected Impact	Risk
S0-01	Set up GitHub Actions CI pipeline with lint, type-check, build, and test stages	**P0**	None	12	Medium	.github/workflows/ci.yml, package.json	Every PR validated automatically; broken builds caught before merge	Medium: Pipeline config may need iteration for monorepo support
S0-02	Configure Vitest test runner with React Testing Library and path alias support	**P0**	None	8	Medium	vitest.config.ts, tsconfig.json, src/test-setup.ts	Test infrastructure operational; developers can write and run tests locally	Medium: Path alias resolution in tests may require custom Vitest config
S0-03	Write 10 critical smoke tests covering auth flow, dashboard load, and API connectivity	**P0**	S0-02	10	Medium	src/__tests__/auth.test.tsx, src/__tests__/dashboard.test.tsx	Core user flows verified automatically; regression protection for critical paths	Low: Smoke tests are isolated and do not modify existing code
S0-04	Fix broken shadcn/ui path alias in tsconfig and component imports	P1	None	6	Easy	tsconfig.json, components.json, src/components/ui/*.tsx	All shadcn components resolve correctly; IDE autocompletion restored	Low: Straightforward path mapping fix with immediate verification
S0-05	Audit and remove unused npm dependencies from package.json	P1	None	4	Easy	package.json, package-lock.json	Reduced install time, smaller node_modules, clearer dependency graph	Medium: Some deps may be dynamically imported; removal needs grep verification
S0-06	Set up Vercel deployment preview on every PR with automatic environment variables	P1	S0-01	6	Easy	vercel.json, .env.example, .github/workflows/ci.yml	Stakeholders can review visual changes per PR; no more manual deploys	Low: Vercel integration is well-documented and mostly configuration
S0-07	Add ESLint strict mode and enforce no-unused-vars, no-explicit-any rules	P1	S0-01	8	Medium	.eslintrc.js, src/**/*.ts, src/**/*.tsx	Code quality enforced automatically; type safety improved across codebase	Medium: May surface many existing violations requiring?? fixes
S0-08	Configure Husky pre-commit hooks with lint-staged for format-on-commit	P2	S0-07	6	Easy	.husky/pre-commit, .lintstagedrc.json, package.json	Consistent code formatting enforced; no more style debates in code review	Low: Pure tooling addition with no impact on existing code behavior

Table 3: Sprint 0 Task Details (8 tasks, 60 hours estimated)

4. Sprint 1: Infrastructure (Weeks 2–3)

Sprint 1 builds the observability and reliability infrastructure that transforms VIXOR from a prototype into a production-ready application. The sprint introduces application monitoring through a structured logging framework, error boundary components that prevent cascading UI failures, and a comprehensive error tracking integration. API versioning is established to enable backward-compatible evolution of the backend interface,

and rate limiting is tuned to protect against abuse while supporting legitimate usage patterns. This sprint also addresses the fragile build configuration by pinning dependency versions and adding build caching, reducing the frequency of broken builds caused by upstream changes.

The monitoring infrastructure introduced in this sprint serves a dual purpose: it provides immediate value by alerting the team to production errors in real time, and it creates the data foundation that informs prioritization decisions in later sprints. For example, error tracking data will reveal which components fail most frequently, guiding the component consolidation work in Sprint 2. Similarly, API usage patterns observed through monitoring will inform the route reduction decisions in Sprint 5. The sprint deliverables include a Grafana dashboard (or equivalent) showing key health metrics, error boundaries wrapping every major page, and a versioned API endpoint that existing clients can continue using without modification.

4.1 Sprint 1 Task Breakdown

ID	Task	Priority	Deps	Hours	Difficulty	Affected Files	Expected Impact	Risk
S1-01	Integrate Sentry error tracking with source map upload and release tracking	**P0**	S0-01	12	Medium	next.config.mjs, src/lib/sentry.ts, src/app/layout.tsx	All runtime errors captured with full stack traces and user context	Medium: Source map upload requires build pipeline coordination
S1-02	Implement React error boundary components for page-level and widget-level fallbacks	P1	None	10	Medium	src/components/ErrorBoundary.tsx, src/app/**/page.tsx	Cascading UI failures prevented; users see graceful fallback instead of blank pages	Low: Error boundaries are additive and do not modify existing component logic
S1-03	Set up structured logging with pino or winston and log level configuration	P1	None	10	Easy	src/lib/logger.ts, src/middleware.ts, next.config.mjs	All application events logged consistently; debugging time reduced significantly	Low: Logging is a cross-cutting concern with minimal integration risk
S1-04	Implement API versioning with /api/v1/ prefix and deprecation headers	P1	None	12	Hard	src/app/api/v1/**/*.ts, src/lib/api-version.ts, src/middleware.ts	API evolution enabled without breaking existing clients; clear migration path	Medium: All API routes must be migrated; requires careful routing configuration
S1-05	Configure rate limiting middleware with per-user and per-IP tiers	P1	None	10	Medium	src/middleware.ts, src/lib/rate-limit.ts, src/app/api/**/*.ts	API abuse prevented; fair usage enforced; backend stability improved	Medium: Rate limits must be calibrated to avoid blocking legitimate high-frequency users
S1-06	Pin dependency versions and add lockfile verification to CI pipeline	P1	S0-01	8	Easy	package.json, package-lock.json, .github/workflows/ci.yml	Build reproducibility guaranteed; no more surprise breakages from upstream updates	Low: Version pinning is a standard practice with well-understood tooling

ID	Task	Priority	Deps	Hours	Difficulty	Affected Files	Expected Impact	Risk
S1-07	Create health check endpoint (/api/health) with database and external service status	P2	S1-03	8	Easy	src/app/api/health/route.ts, src/lib/health.ts	Infrastructure monitoring enabled; automated alerting on service degradation	Low: Self-contained endpoint with minimal dependencies on existing code
S1-08	Add performance monitoring with Web Vitals tracking and Core Web Vitals LCP/FID/CLS	P2	S1-01	10	Medium	src/lib/web-vitals.ts, src/app/layout.tsx, src/components/ReportWebVitals.tsx	Performance regressions detected automatically; UX degradation quantified	Medium: Web Vitals reporting requires careful threshold calibration

Table 4: Sprint 1 Task Details (8 tasks, 80 hours estimated)

5. Sprint 2: Component Refactoring (Weeks 4–5)

Sprint 2 tackles the most significant maintainability issues in the component layer. The AppShell component, at 1,688 lines of code, is a monolithic structure that handles navigation, sidebar state, theme switching, notification display, and responsive layout all within a single file. This component must be decomposed into focused sub-components with clear responsibilities, each independently testable and reusable. Simultaneously, seven chart variants scattered across the codebase must be consolidated into a unified ChartCard component that accepts a configuration prop to determine the visualization type. This consolidation eliminates hundreds of lines of duplicated rendering logic and ensures visual consistency across all data displays.

The sprint also introduces shared error and loading state components that replace the ad-hoc implementations currently scattered throughout the application. Currently, each page implements its own loading spinner and error message, leading to visual inconsistency and duplicated effort. By extracting these into shared components with standardized APIs, the team ensures that every page presents a cohesive experience during data fetching states. The component audit identified 18 problems, 10 of which are P1, meaning this sprint addresses the majority of component-level technical debt in a single focused effort. The risk in this sprint is primarily around regression: splitting the AppShell requires careful testing to ensure no navigation state is lost during the decomposition.

5.1 Sprint 2 Task Breakdown

ID	Task	Priority	Deps	Hours	Difficulty	Affected Files	Expected Impact	Risk
S2-01	Decompose AppShell into Sidebar, TopBar, ThemeToggle, NotificationPanel sub-components	P1	S0-03	16	Hard	src/components/layout/AppShell.tsx, src/components/layout/Sidebar.tsx, src/components/layout/TopBar.tsx	AppShell reduced from 1,688 to under 200 LOC; each sub-component independently testable	High: Navigation state management must be preserved; extensive manual QA required

ID	Task	Priority	Deps	Hours	Difficulty	Affected Files	Expected Impact	Risk
S2-02	Create unified ChartCard component with configurable chart type prop (line, bar, area, pie, scatter, candlestick, heatmap)	P1	None	16	Hard	src/components/charts/ChartCard.tsx, src/components/charts/configs/*.ts	Seven chart variants replaced by one component; visual consistency guaranteed	High: Each chart type has unique data transform needs; thorough visual QA per type
S2-03	Build shared LoadingState component with skeleton, spinner, and progress variants	P1	None	8	Easy	src/components/shared/LoadingState.tsx, src/components/shared/Skeleton.tsx	Consistent loading UX across all pages; reduced per-page implementation effort	Low: New component with no impact on existing page logic until adopted
S2-04	Build shared ErrorState component with retry action and error detail expansion	P1	None	8	Easy	src/components/shared/ErrorState.tsx, src/components/shared/RetryButton.tsx	Consistent error UX; users always have a clear path to recovery	Low: Additive component that pages opt into during their own refactoring
S2-05	Build shared EmptyState component with illustration, message, and CTA slot	P1	None	6	Easy	src/components/shared/EmptyState.tsx	Empty data scenarios handled gracefully; users guided to next action	Low: Pure presentational component with no side effects
S2-06	Migrate all pages to use shared LoadingState, ErrorState, and EmptyState	P2	S2-03, S2-04, S2-05	12	Medium	src/app/**/page.tsx (all route pages)	Visual consistency across entire application; reduced code duplication	Medium: Each page may have unique edge cases requiring custom handling
S2-07	Extract duplicated hooks (useLocalStorage, useDebounce, useMediaQuery) into shared hooks directory	P1	None	8	Easy	src/hooks/useLocalStorage.ts, src/hooks/useDebounce.ts, src/hooks/useMediaQuery.ts	Duplicated hook logic eliminated; single source of truth for each utility	Low: Hooks can be migrated incrementally one consumer at a time
S2-08	Add component-level tests for AppShell sub-components and ChartCard with all variants	P2	S2-01, S2-02	14	Medium	src/__tests__/layout/*.test.tsx, src/__tests__/charts/*.test.tsx	Component regression protection; confidence in refactoring correctness	Low: Tests are isolated and serve as documentation for component behavior

Table 5: Sprint 2 Task Details (8 tasks, 88 hours estimated)

6. Sprint 3: Domain Cleanup (Weeks 6–7)

Sprint 3 addresses the domain model, which the domain audit identified as containing 43 problems across 23 domain modules. The most critical finding is three P0 issues that affect data integrity, including missing unique constraints on user-facing identifiers, inconsistent timestamp handling between client and server, and a race condition in the portfolio aggregation logic that can produce incorrect balance calculations. These P0 issues must be resolved before any domain consolidation work begins, as they represent correctness bugs that could lead to financial miscalculations in a trading application. The sprint also addresses the 22 P1 problems, which primarily involve inconsistent naming conventions across domains, missing input validation on domain operations, and redundant data transformations that should be pushed to the repository layer.

A significant portion of this sprint is dedicated to consolidating the discover domain, which the audit found to be fragmented across five separate modules that perform overlapping discovery and search operations. These modules will be merged into a single unified discovery domain with clear boundaries between search, filter, and recommendation responsibilities. Additionally, the dead copilot v1 code, which was superseded by copilot v2 but never removed, will be fully excised from the codebase. This dead code represents approximately 1,200 lines across 15 files and creates confusion for new developers who encounter references to the old copilot architecture. Domain-level tests will be introduced to verify the correctness of the consolidated domain logic and prevent regressions.

6.1 Sprint 3 Task Breakdown

ID	Task	Priority	Deps	Hours	Difficulty	Affected Files	Expected Impact	Risk
S3-01	Fix P0 data integrity: add unique constraints on user identifiers and portfolio names	**P0**	None	8	Medium	prisma/schema.prisma, src/lib/db/migrations/*.sql	Data integrity guaranteed; duplicate user/portfolio creation prevented at database level	Medium: Migration must handle existing duplicate data before constraint can be applied
S3-02	Fix P0 timestamp consistency: unify client and server time handling to UTC with ISO 8601	**P0**	None	10	Medium	src/lib/datetime.ts, src/domains/**/*.ts, src/app/api/**/*.ts	Timezone bugs eliminated; all timestamps display consistently regardless of user location	Medium: All date comparisons and display logic must be audited for timezone assumptions
S3-03	Fix P0 race condition in portfolio aggregation with optimistic locking or serializable transactions	**P0**	S3-01	14	Expert	src/domains/portfolio/aggregation.ts, src/lib/db/transactions.ts	Correct balance calculations guaranteed even under concurrent updates	**High: Financial correctness is critical; requires extensive load testing**
S3-04	Consolidate 5 fragmented discover modules into a unified discovery domain	P1	S3-01	16	Hard	src/domains/discovery/**/*.ts, src/domains/discover-*.ts (remove)	Domain boundaries clarified; 5 modules become 1 with clear responsibilities	High: Import paths change across codebase; requires full regression testing
S3-05	Remove dead copilot v1 code (15 files, ~1,200 lines) and update all references to v2	P1	None	8	Easy	src/domains/copilot-v1/** (delete), src/domains/copilot/**/*.ts	Dead code eliminated; no confusion between copilot versions for developers	Low: Dead code by definition has no runtime callers; removal is safe
S3-06	Standardize domain naming conventions and add input validation to all domain operations	P1	S3-04	12	Medium	src/domains/**/*.ts, src/lib/validation/schemas.ts	Consistent domain API; invalid inputs rejected early with clear error messages	Medium: Validation rules must be agreed upon with product team
S3-07	Write domain-level unit tests for all 23 domain modules with minimum 80% branch coverage	P2	S3-04, S3-06	16	Medium	src/__tests__/domains/**/*.test.ts	Domain logic regression protection; confidence in business rule correctness	Low: Tests exercise existing behavior; no modifications to domain logic required

Table 6: Sprint 3 Task Details (7 tasks, 84 hours estimated)

7. Sprint 4: UX Improvements (Weeks 8–10)

Sprint 4 focuses on the user-facing experience, addressing 47 UI/UX problems identified across the application. The two critical findings are accessibility violations that must be resolved for WCAG 2.1 AA compliance: missing ARIA labels on interactive chart elements and insufficient color contrast ratios in the sidebar navigation. These violations pose not only a legal compliance risk but also exclude users with visual impairments from using the application effectively. Beyond accessibility, the sprint addresses significant mobile responsiveness gaps where key pages such as the dashboard, portfolio view, and settings page are either partially or completely broken on viewports narrower than 768 pixels. Given the increasing proportion of users accessing web applications from mobile devices, these gaps represent a significant barrier to user adoption and retention.

The sprint also introduces comprehensive empty and loading states across all data-driven pages. Currently, many pages render nothing when data is unavailable, leaving users uncertain whether the page is broken, loading, or simply empty. The shared components introduced in Sprint 2 (EmptyState, LoadingState, ErrorState) provide the building blocks, and this sprint focuses on integrating them into every page with contextually appropriate messaging and calls to action. Onboarding improvements are also included, with a guided first-run experience that walks new users through connecting an exchange, setting up their first portfolio, and understanding the dashboard layout. The current onboarding drops users directly into an empty dashboard with no guidance, which is a primary driver of the poor retention metrics identified in the product audit.

7.1 Sprint 4 Task Breakdown

ID	Task	Priority	Deps	Hours	Difficulty	Affected Files	Expected Impact	Risk
S4-01	Fix critical a11y: add ARIA labels to all interactive chart elements and ensure screen reader compatibility	**P0**	S2-02	14	Medium	src/components/charts/ChartCard.tsx, src/components/charts/**/*.tsx	Charts accessible to screen reader users; WCAG 2.1 AA compliance for data visualization	Medium: Complex chart interactions require careful ARIA live region management
S4-02	Fix critical a11y: resolve color contrast violations in sidebar, buttons, and form labels	**P0**	None	10	Easy	src/components/layout/Sidebar.tsx, tailwind.config.ts, src/app/globals.css	All text meets WCAG AA contrast ratio of 4.5:1; improved readability for all users	Low: Color token changes with automated contrast verification
S4-03	Implement responsive layouts for dashboard, portfolio, and settings pages (mobile-first)	P1	S2-01	18	Hard	src/app/dashboard/page.tsx, src/app/portfolio/page.tsx, src/app/settings/page.tsx	Full functionality on mobile devices; expanded addressable user base	High: Complex data tables and charts require significant layout rethinking for mobile
S4-04	Integrate EmptyState, LoadingState, and ErrorState into all data-driven pages with contextual messaging	P1	S2-06	14	Medium	src/app/**/page.tsx (all data pages)	No page ever shows blank; users always understand current state and next action	Medium: Each page needs unique messaging and CTAs tailored to its context

ID	Task	Priority	Deps	Hours	Difficulty	Affected Files	Expected Impact	Risk
S4-05	Design and implement guided onboarding flow with exchange connection, portfolio setup, and dashboard tour	P1	S4-03, S4-04	18	Hard	src/app/onboarding/page.tsx, src/components/onboarding/*.tsx, src/lib/onboarding.ts	New user activation rate improved; time-to-value reduced from undefined to under 5 minutes	Medium: Onboarding flow requires product input on optimal step sequence and messaging
S4-06	Add keyboard navigation support and focus management across all interactive components	P1	S4-01	12	Medium	src/components/**/*.tsx, src/app/globals.css	Full keyboard operability; improved accessibility for power users and assistive technology	Medium: Focus trap implementation in modals and dropdowns requires careful state management

Table 7: Sprint 4 Task Details (6 tasks, 86 hours estimated)

8. Sprint 5: Feature Rationalization (Weeks 11–14)

Sprint 5 is the most consequential sprint in terms of product impact, reducing the application from 39 routes to 12 core routes. The product audit identified that the current route count is approximately three times what an MVP should contain, with many routes serving aspirational features that are either partially implemented, completely empty, or duplicate functionality available elsewhere. This feature creep dilutes engineering effort, confuses users with an overwhelming navigation structure, and increases the attack surface for bugs and accessibility issues. The rationalization process follows a data-driven approach: routes are ranked by user engagement metrics (from the monitoring infrastructure set up in Sprint 1), business value, and implementation completeness. Routes that score low across all three dimensions are candidates for removal, while high-value routes receive investment for polish and completion.

The 12 core routes selected for retention represent the essential user journey: authentication, dashboard overview, portfolio management, market data, trading execution, settings, and a small number of supporting pages for help and documentation. The 27 removed routes will be handled through a combination of full deletion (for empty or broken pages), redirection to the nearest core route (for partially implemented features), and archival in a separate branch for potential future revival. This sprint also addresses the missing retention systems identified in the product audit, including email notification preferences, activity feed persistence, and user preference storage. The sprint is allocated four weeks, the longest of any sprint, because feature reduction is inherently risky from a user perspective and requires careful communication, migration support, and the option for users to provide feedback on removed features before they are permanently deleted.

8.1 Sprint 5 Task Breakdown

ID	Task	Priority	Deps	Hours	Difficulty	Affected Files	Expected Impact	Risk
S5-01	Analyze route usage metrics and rank all 39 routes by engagement, value, and completeness	P1	S1-01	12	Medium	src/app/**/page.tsx (all routes), analytics data	Data-driven route prioritization; defensible decisions on what to keep vs. remove	Low: Analysis task with no code changes
S5-02	Remove 15 empty or broken routes and set up redirects to nearest core route	P1	S5-01	16	Medium	src/app/[removed-routes]/** (delete), next.config.mjs (redirects)	Dead weight eliminated; users redirected seamlessly; navigation simplified	Medium: External links and bookmarks may break; redirect coverage must be comprehensive
S5-03	Archive 12 aspirational routes to feature/rationalization branch with documentation	P1	S5-01	10	Easy	src/app/[aspirational]/** (move to branch)	Aspirational code preserved for future use; main branch cleaned of incomplete features	Low: Code is moved, not deleted; can be restored from Git history or branch
S5-04	Simplify navigation to expose only 12 core routes in sidebar and header	P1	S5-02, S5-03, S2-01	12	Medium	src/components/layout/Sidebar.tsx, src/components/layout/TopBar.tsx, src/lib/navigation.ts	Navigation clarity dramatically improved; users find features faster; cognitive load reduced	Medium: Navigation grouping and labeling requires UX input and user testing
S5-05	Implement user preference storage system with persistence across sessions	P2	S3-01	14	Medium	src/domains/user/preferences.ts, src/app/api/user/preferences/route.ts, prisma/schema.prisma	User settings persist across sessions; improved personalization; reduced friction on return visits	Low: Standard CRUD with localStorage fallback; well-understood pattern
S5-06	Implement basic retention systems: email digest preferences and activity feed	P2	S5-05	18	Hard	src/domains/notifications/**/*.ts, src/app/api/notifications/**/*.ts, src/components/ActivityFeed.tsx	User engagement improved through proactive communication; return visit rate increased	High: Email delivery infrastructure requires third-party integration (SendGrid/Resend)

Table 8: Sprint 5 Task Details (6 tasks, 82 hours estimated)

9. Risk Assessment Matrix

Every refactoring effort carries inherent risk, and this plan identifies and mitigates the most significant risks across all six sprints. The risk matrix below categorizes risks by likelihood and impact, with specific mitigation strategies for each. The highest-risk items are the AppShell decomposition (S2-01) and the portfolio aggregation race condition fix (S3-03), both of which involve complex state management with potential for data loss if implemented incorrectly. The feature rationalization in Sprint 5 carries user-facing risk, as removing routes that any user depends on creates immediate dissatisfaction. Each high-risk task includes specific mitigation strategies, and the CI/CD pipeline established in Sprint 0 provides the safety net of automated regression testing that catches issues before they reach production.

Risk	Likelihood	Impact	Mitigation Strategy
AppShell decomposition breaks navigation state	Medium	**High**	Extensive E2E tests before merge; feature flag for gradual rollout; rollback plan
Portfolio race condition fix causes performance regression	Low	**High**	Load testing with production-like data; benchmark before/after; canary deployment
Route removal breaks existing user bookmarks	High	Medium	Comprehensive 301 redirects; 30-day grace period with deprecation notices in UI
Chart consolidation misses edge case rendering	Medium	Medium	Visual regression testing with Percy/Chromatic; per-chart-type test suites
Domain consolidation introduces circular dependencies	Medium	Medium	Dependency graph analysis tool in CI; strict module boundary linting rules
A11y fixes conflict with design system	Low	Low	Design tokens updated centrally; contrast checker integrated into CI pipeline
CI pipeline false positives block legitimate PRs	Medium	Low	Flaky test detection and quarantine; tiered check requirements (warn vs. block)
Monitoring overhead degrades application performance	Low	Low	Sampling-based metrics collection; performance budget alerts; lazy-loaded SDK

Table 9: Risk Assessment Matrix with Mitigation Strategies

10. Resource Estimation and Timeline

The total estimated effort across all six sprints is 480 hours of focused engineering time. This estimate includes implementation, testing, code review, and documentation for each task, but excludes product design time, stakeholder review cycles, and deployment verification. A single senior engineer working full-time on this plan would complete it in approximately 12 weeks, which aligns with the 14-week sprint timeline that includes buffer for unexpected complexity and review cycles. With two engineers, the timeline can be compressed to approximately 8 weeks, as most sprint-internal tasks can be parallelized. With three engineers, the theoretical minimum is 6 weeks, though coordination overhead and merge conflicts make this unrealistic without very careful task allocation.

Configuration	Sprint Duration	Total Duration	Parallelism	Recommended For
1 Senior Engineer	1–4 weeks per sprint	**12–14 weeks**	None	Solo founder or small team
2 Engineers	0.5–2 weeks per sprint	**8–10 weeks**	Medium	Small startup team
3 Engineers	0.5–1.5 weeks per sprint	**6–8 weeks**	High	Dedicated refactoring team

Table 10: Resource Configuration Options

The recommended approach is a two-engineer configuration, where one engineer focuses on infrastructure and backend concerns (Sprints 0, 1, and 3) while the other handles frontend component and UX work (Sprints 2 and 4). Sprint 5 (feature rationalization) requires both engineers plus product input, as the decisions about which routes to keep have business implications that extend beyond technical considerations. Regardless of

team size, the sprint sequence should not be reordered, as each sprint builds on the deliverables of its predecessors. The buffer built into the 14-week timeline accounts for the inevitable discovery of additional issues during implementation, as well as the time required for thorough code review and stakeholder feedback on user-facing changes.

11. Success Criteria and Exit Metrics

The refactor plan will be considered successful when all of the following criteria are met, measured at the end of Sprint 5. These criteria are designed to be objectively verifiable and represent a meaningful improvement in the health, maintainability, and user experience of the VIXOR application. Each criterion maps to one or more audit findings, ensuring complete coverage of the 221 identified problems.

#	Criterion	Measurement	Audit Source
1	CI/CD pipeline passes on 100% of commits to main	GitHub Actions success rate	Architecture (P0)
2	Test coverage exceeds 60% for all refactored modules	Vitest coverage report	Architecture (P0)
3	Zero P0 domain integrity issues remain open	Code review + load testing	Domain (P0)
4	AppShell component under 200 LOC	Lines of code count	Component (P1)
5	Single ChartCard component replaces all 7 chart variants	Code audit: chart components count	Component (P1)
6	WCAG 2.1 AA compliance with zero critical violations	axe-core automated audit	UI/UX (P0)
7	All core pages responsive down to 375px viewport	Manual testing on mobile devices	UI/UX (P1)
8	Route count reduced from 39 to 12 or fewer	File system count of page.tsx	Product (P1)
9	Production error rate below 0.1% of sessions	Sentry error tracking dashboard	Technical (P0)
10	Lighthouse performance score above 85	Lighthouse CI audit	Technical (P1)

Table 11: Success Criteria with Objective Measurements

These ten criteria represent the minimum bar for a successful refactor. Achieving all ten confirms that the most critical problems identified across all six audits have been resolved and that the application is measurably healthier than before the refactor began. Additional stretch goals include achieving 80% test coverage (versus the 60% minimum), reducing the largest component to under 100 LOC, and achieving a Lighthouse score above 90 across all core pages. Progress toward these criteria should be tracked weekly during sprint retrospectives, with any criterion at risk of not being met escalated immediately for resolution.

12. Appendix: Full Task Index

The following table provides a complete index of all 43 tasks across all six sprints, sorted by sprint and task ID. This index serves as a quick reference for project managers and engineers to locate specific tasks and their

attributes without scrolling through the detailed sprint sections above. Each task includes its ID, a brief description, priority level, estimated hours, and difficulty rating. Dependencies are noted where applicable, and the sprint column indicates when the task is scheduled for execution.

ID	Task Summary	Priority	Hours	Diff.	Sprint
S0-01	GitHub Actions CI pipeline	**P0**	12	Medium	Sprint 0
S0-02	Vitest test runner setup	**P0**	8	Medium	Sprint 0
S0-03	10 critical smoke tests	**P0**	10	Medium	Sprint 0
S0-04	Fix shadcn/ui path alias	P1	6	Easy	Sprint 0
S0-05	Remove unused dependencies	P1	4	Easy	Sprint 0
S0-06	Vercel deployment previews	P1	6	Easy	Sprint 0
S0-07	ESLint strict mode	P1	8	Medium	Sprint 0
S0-08	Husky pre-commit hooks	P2	6	Easy	Sprint 0
S1-01	Sentry error tracking	**P0**	12	Medium	Sprint 1
S1-02	React error boundaries	P1	10	Medium	Sprint 1
S1-03	Structured logging	P1	10	Easy	Sprint 1
S1-04	API versioning /api/v1/	P1	12	Hard	Sprint 1
S1-05	Rate limiting middleware	P1	10	Medium	Sprint 1
S1-06	Pin dependency versions	P1	8	Easy	Sprint 1
S1-07	Health check endpoint	P2	8	Easy	Sprint 1
S1-08	Web Vitals monitoring	P2	10	Medium	Sprint 1
S2-01	AppShell decomposition	P1	16	Hard	Sprint 2
S2-02	Unified ChartCard component	P1	16	Hard	Sprint 2
S2-03	Shared LoadingState	P1	8	Easy	Sprint 2
S2-04	Shared ErrorState	P1	8	Easy	Sprint 2
S2-05	Shared EmptyState	P1	6	Easy	Sprint 2
S2-06	Migrate pages to shared states	P2	12	Medium	Sprint 2
S2-07	Extract duplicated hooks	P1	8	Easy	Sprint 2
S2-08	Component-level tests	P2	14	Medium	Sprint 2
S3-01	P0 unique constraints	**P0**	8	Medium	Sprint 3
S3-02	P0 timestamp unification	**P0**	10	Medium	Sprint 3
S3-03	P0 race condition fix	**P0**	14	Expert	Sprint 3
S3-04	Consolidate discover domains	P1	16	Hard	Sprint 3
S3-05	Remove dead copilot v1	P1	8	Easy	Sprint 3
S3-06	Domain naming and validation	P1	12	Medium	Sprint 3
S3-07	Domain-level tests	P2	16	Medium	Sprint 3

ID	Task Summary	Priority	Hours	Diff.	Sprint
S4-01	A11y: chart ARIA labels	**P0**	14	Medium	Sprint 4
S4-02	A11y: color contrast fix	**P0**	10	Easy	Sprint 4
S4-03	Responsive layouts (mobile)	P1	18	Hard	Sprint 4
S4-04	Integrate shared states	P1	14	Medium	Sprint 4
S4-05	Guided onboarding flow	P1	18	Hard	Sprint 4
S4-06	Keyboard navigation	P1	12	Medium	Sprint 4
S5-01	Route usage analysis	P1	12	Medium	Sprint 5
S5-02	Remove 15 empty routes	P1	16	Medium	Sprint 5
S5-03	Archive 12 aspirational routes	P1	10	Easy	Sprint 5
S5-04	Simplify navigation	P1	12	Medium	Sprint 5
S5-05	User preference storage	P2	14	Medium	Sprint 5
S5-06	Retention systems	P2	18	Hard	Sprint 5

Table 12: Complete Task Index (43 tasks, ~480 hours total)

This Refactor Plan represents a comprehensive, methodical approach to transforming the VIXOR codebase from its current state into a production-ready, maintainable, and accessible application. By following the sprint sequence and adhering to the success criteria defined above, the team can systematically eliminate technical debt while maintaining business continuity and delivering incremental value at each sprint boundary. The plan is designed to be adaptable: if external factors require acceleration or deceleration, the granular task breakdown allows for selective execution without losing the coherence of the overall strategy. Regular retrospectives at the end of each sprint will provide opportunities to adjust estimates, reprioritize tasks, and incorporate lessons learned into subsequent sprints.